

为什么数据分析PA和数探查数这么准

从 Text-to-SQL 的普遍困境到知识工程驱动的解法——一次数据分析能力的技术平权

一、引子：一个反直觉的事实

在飞书群里 @数探，说一句"帮我看看 eufy 上周美国的订单销售额"，十几秒后，一张带图表的数据卡片就回来了——数字是对的，口径是对的，去重逻辑也是对的。

同样的问题丢给市面上任何一个通用 AI 助手，大概率会得到一条语法正确但业务含义全错的 SQL。

区别在哪里？

模型没变，都是同一个 Claude。 变的是我们在 AI 和数据库之间，构建了一层机器可读的业务知识层。不是因为 AI 模型有多聪明，而是我们把业务规则写透了，并且用 AI 能理解的方式组织起来。

核心就一句话：把人查数的思考过程，变成了机器可执行的规则链。

二、Text-to-SQL 为什么这么难？——行业普遍痛点

2.1 AI 不是不会写 SQL，是不懂你的业务

2026 年的大语言模型，SQL 语法能力已经不是瓶颈。GPT、Claude、Gemini 都能写出语法完美的 SQL。

但"语法对、业务错"才是真正的杀手。

举个例子：统计订单数量，AI 会很自然地写出：

代码块

```
1  -- 语法完美，业务全错
2  COUNT(DISTINCT order_id) AS order_count
```

这条 SQL 在绝大多数教科书里都是标准写法。但在我们的业务场景下，它是**错的**——因为不同品牌、不同国家的 order_id 可能重复，直接去重会少算。正确的写法是：

代码块

```
1  -- 三段拼接去重，业务正确
2  COUNT(DISTINCT CONCAT(brand, '_', country, '_', order_id)) AS order_count
```

这不是 SQL 语法问题，是业务规则问题。**AI 不知道这条规则，就不可能写对。**

2.2 四个致命的"不知道"

我们观察了大量 Text-to-SQL 的失败案例，归纳出 AI 查数失准的四个根因：

#	不知道什么	后果
1	不知道该查哪张表	公司可能有几十张表，AI 靠表名猜，猜错了后面全白干
2	不懂字段的业务语义	brand 和 shop_brand 看起来差不多，实际上一个是 SKU 品牌，一个是店铺品牌
3	不知道隐性业务规则	brand 必须小写、country 必须大写、转化率不能用 AVG——这些规则不写在表结构里
4	不知道什么时候该问用户	用户说"查 SKU 数据"，到底是 bundle_sku（组合包）还是 sku（单品）？不问就猜，猜错概率极高

这四个"不知道"，DESCRIBE TABLE 一个都解决不了。**表结构只有骨架，没有灵魂。**

2.3 为什么通用 RAG/向量检索在这里效果有限？

很多团队的第一反应是："上 RAG 不就行了？把文档丢进向量数据库，让 AI 自己检索。"

但在 Text-to-SQL 场景下，向量检索有一个本质局限：**它擅长"找相关内容"，但业务规则需要的是"精确指令"。**

"brand 必须用 lower() 转小写"——这条规则的向量表示和用户问"eufy 的销售额"之间的语义距离并不近。向量检索可能返回一堆相关文档片段，但不保证命中这条关键规则。

我们需要的不是模糊匹配，而是：**当 AI 准备写 brand 相关的 SQL 时，这条规则必须被强制加载，一次都不能漏。**

三、不同规模下的最优解

认识到问题之后，解法其实和表的数量强相关。

3.1 表的规模决定架构选型

表数量	推荐方案	原因
10 张以内	直接写 10 个表文档，路径写死给 AI	全量加载也才几千 token，简单直接

20-40 张（我们的情况）	多层索引架构：先用 JSON 索引定位，再深度读取表文档	全量加载太浪费 context，但表不够多不值得上重基建
50 张以上	企业级向量索引数据库 + 知识图谱	人工维护索引的成本开始超过自动化方案

我们的业务范围大约在 20-40 张核心表。这个量级有一个特点：**不大不小，但每张表都有独特的业务规则。**

全量加载 20+ 份文档（每份 1000-3000 字）会浪费大量 context window，但上一套完整的向量数据库又过于笨重。

3.2 为什么"渐进式披露"在这个规模下是最优解？

我们找到的答案是**渐进式披露（Progressive Disclosure）**——AI 不需要一开始就知道所有表的所有细节，而是按需、分层加载：

代码块

```
1  用户需求进来
2      |
3      v
4  第一层：读 tables_metadata.json (JSON 语义索引)
5      通过 keywords / business_terms / use_cases 匹配
6      快速锁定 1-2 张目标表
7      |
8      v
9  第二层：读 tables/{表名}.md (完整表文档 + AI_HINT)
10     获取字段定义、业务规则、澄清触发器、SQL 示例
11     |
12     v
13  第三层（按需）：读 relationships.json (跨表关系)
14     需要 JOIN 时才加载，获取关联条件和禁忌规则
```

这个设计的核心原则是：**由模糊到具体，循序渐进。** 第一层用几百 token 完成表定位，第二层才花 1000-2000 token 深入理解，避免无关表的信息污染 AI 的判断。

一个关键的设计洞察：**给 AI 用 JSON（结构化、精确匹配），给人用 Markdown（可读、好浏览）。** 同一份知识资产，针对不同读者用不同格式呈现。我们有一份 tables_overview.md 按业务分类组织所有表的概述——这是给人浏览的入口；AI 的执行链路上不走这个文件，而是直接读 JSON 索引做精确匹配。

四、Data Catalog：把分析师的脑子写成 AI 能懂的手册

4.1 起点：按分析师的思路去做事

理解 Data Catalog 的设计，先要理解一个有经验的数据分析师是怎么工作的：

代码块

```
1  用户需求: "帮我看看 eufy 最近一周各国家的转化率"
2      |
3  1 理解需求 - 转化率 + eufy + 一周 + 分国家
4      |
5  2 找表 - 转化率应该在 cr_details_aggregate
6      |
7  3 确认规则 - brand 小写、转化率用 SUM/SUM 不能用 AVG
8      |
9  4 澄清需求 - eufy 有 3 个业务线，看全部还是某个？
10      | 上周是自然周还是海翼周？
11      |
12  5 写 SQL - 按规则写出正确查询
13      |
14  6 交付结果 - 执行、总结、可视化
```

这个过程中，真正让查数"准"的不是写 SQL 的技术能力，而是步骤2/3/4中积累的业务知识。

Data Catalog 做的事情，就是把这套隐性知识，用 AI 可读的结构化格式沉淀下来。

4.2 两层检索架构

代码块

```
1  .claude/data_catalog/
2  |-- tables_overview.md          <- 人的浏览入口（按业务分类）
3  |-- metadata/
4  |   |-- tables_metadata.json    <- AI 的第一站：语义索引
5  |   |-- relationships.json     <- 跨表 JOIN 规则（按需读取）
6  |-- tables/
7  |   |-- shopify_order_sku_wide.md <- AI 的第二站：深度文档
8  |   |-- cr_details_aggregate.md
9  |   |-- ...（共 20+ 张表）
```

AI 的检索链路：

第一层：tables_metadata.json —— 从用户语言到目标表

JSON 格式的语义索引，每张表的条目包含：

字段	作用	示例
keywords	关键词匹配	["订单", "order", "GMV", "销售额"]

business_terms	业务术语映射	["订单数量","复购率","客单价"]
use_cases	典型使用场景	"统计某品牌某国家的日销售额趋势"
exclude_patterns	排除误匹配	用户说"会员"时，不要误选订单表
critical_business_rules	核心规则精简版	"订单去重必须 CONCAT(brand,country,order_id)"

用户说"查一下转化率"，AI 在 JSON 索引中匹配 keywords 和 business_terms，精准命中 cr_details_aggregate，而不是去猜。

第二层：tables/{表名}.md + AI_HINT —— 深度理解业务规则

锁定目标表后，AI 读取完整的表文档，获取字段定义、业务规则、澄清指引和 SQL 示例。

4.3 AI_HINT：让 AI 像有经验的分析师一样遵守规则

这是整个 Data Catalog 最关键的设计创新。

每张表文档的顶部，用 HTML 注释格式嵌入了一组机器可读的标签：

代码块

- 1 AI_HINT: business_rule_case=brand必须用lower()转小写,country必须大写
- 2 AI_HINT: business_rule_order_dedup=统计订单数必须 CONCAT(brand,'_',country,'_',order_id)去重
- 3 AI_HINT: ask_user_about_sku=用户提到sku时,必须澄清是bundle_sku还是sku
- 4 AI_HINT: auto_mapping_prime=付费会员->prime_type in ('年费会员','月费会员')

为什么用 HTML 注释？因为它对人"隐形"（不干扰 Markdown 渲染），但对 AI 完全可见。人看到的是干净的文档，AI 看到的是结构化的指令。一份文档，同时服务两类读者。

AI_HINT 的三种类型

类型	触发时机	AI 的行为
business_rule	写 SQL 时	强制执行，不问用户。如：brand 小写、订单三段去重、转化率用 SUM/SUM
ask_user_about	识别到歧义关键词时	必须追问用户确认。如：SKU 粒度、渠道口径、会员类型
auto_mapping	识别到业务术语时	自动翻译，不打扰用户。如："安防"->bg='Security'，"付费会员"->prime_type IN (...)

这三种标签形成了一个优先级体系：

- 1. CRITICAL（必须执行）：required_filters、business_rule_* —— SQL 中必须包含，漏了就是错的
- 2. ASK（必须确认）：ask_user_about_* —— 不确定就问，宁可多问一句，不可写错一行
- 3. AUTO（自动处理）：auto_mapping_* —— AI 默默翻译，不打扰用户

正反示例：为什么要"画出禁区"

每条规则都配有正确和错误写法的对比：

代码块

```
1  -- 正确：先求和再相除（加权平均）
2  SUM(orders) / SUM(sessions) AS conversion_rate
3
4  -- 错误：直接平均，会给小流量日期过高权重
5  AVG(orders / sessions) AS conversion_rate
```

为什么不光写正确写法，还要写错误写法？因为 AI 在生成 SQL 时，如果只看到正确写法，仍可能基于训练数据中的常见模式写出错误变体。反面示例相当于在 AI 的决策空间中**画出禁区**——"这个区域不要踏入"。

需求澄清：不是 AI 猜对了，是 AI 问对了

以订单表 shopify_order_sku_wide 为例，我们设置了 7 个澄清触发器：

用户说了什么	为什么有歧义	AI 必须问的问题
"SKU"	bundle_sku vs sku	要组合包粒度还是单品粒度？
"品类"	pdt_name vs sub_category_name	大品类还是子品类？
"品牌"	brand vs shop_brand	产品品牌还是店铺品牌？
"会员"	普通会员 vs 付费会员	全部会员还是只看付费会员？
"渠道"	4 个渠道字段	一级/二级/utm_source/utm_medium？
"站点"	多站点共存	官网还是 APP？
"产品"	名称模糊匹配	请提供具体 SKU 编码

"准"的第一层含义：不是 AI 猜对了，而是 AI 在不确定时主动问对了。

4.4 跨表关系图谱：relationships.json

真实的数据分析经常需要关联多张表。relationships.json 为 AI 提供了完整的 JOIN 说明书：

代码块

```
1  {
2    "relationship_id": "member_to_order",
3    "join_type": "LEFT JOIN",
4    "join_conditions": [
5      {"left_field": "email_sha256", "right_field": "email_hash_sha256"},
6      {"left_field": "brand", "right_field": "brand"}
7    ],
8    "additional_notes": [
9      "禁止使用 country 字段连表",
10     "原因：Shopify 站点覆盖多个国家，用 country 会丢失跨境订单"
11   ]
12 }
```

注意那条「禁止使用 country 字段连表」。直觉上觉得加 country 更精确，但实际上一个 Shopify 站点（如 EU 站）同时服务多个国家，加了 country 反而会丢数据。

这种“哪些不能做”的知识，才是真正的业务护城河。也是 AI 最容易犯错的地方——因为“加更多条件让 JOIN 更精确”是 AI 的训练偏好。

4.5 一个查询的完整旅程

用一个真实案例走完全链路：

代码块

```
1  用户："帮我看看 eufy 上周各国家的转化率"
2
3  == Step 1: 表定位 (tables_metadata.json) ==
4  关键词匹配："转化率" -> keywords 命中 cr_details_aggregate
5  use_cases 包含 "对比品牌转化率"
6  -> 锁定目标表
7
8  == Step 2: 读取表文档 (cr_details_aggregate.md) ==
9  解析 AI_HINT：
10  business_rule_conversion_rate = SUM/SUM, 不能 AVG
11  business_rule_case = brand 小写
12  required_filter = date 必须过滤
13
14  == Step 3: 需求澄清 ==
15  AI："eufy 有 3 个业务线 (eufyclean/eufyhome/eufylife),
16     看全部还是某个具体业务线？"
```

```
17 AI: "上周是指自然周还是海翼周？"
18 用户: "全部，自然周"
19
20 == Step 4: 跨表检查 (relationships.json) ==
21 需要 date_dim 获取 year_week -> JOIN 条件: date <-> date_day
22
23 == Step 5: 生成 SQL ==
```

代码块

```
1  SELECT
2      d.year_week,
3      c.country,
4      SUM(c.orders) / SUM(c.sessions) AS cvr -- 规则: SUM/SUM
5  FROM dtc.ai.cr_details_aggregate c
6  LEFT JOIN dtc.common.date_dim d
7      ON c.date = d.date_day -- 关系: 来自 relationships.json
8  WHERE c.brand = 'eufy' -- 规则: 小写
9      AND c.date >= date_sub(current_date(), 14) -- 约束: 日期必过滤
10     AND d.year_week = (...)
11  GROUP BY d.year_week, c.country
12  ORDER BY cvr DESC
```

每一行 SQL 背后，都有明确的知识来源。不是 AI "感觉"应该这么写，而是文档规定了这么写。

五、数据分析 PA vs 数探：同源不同形

数据分析 PA 和数探是两个不同的产品形态，但它们共享同一套 Data Catalog 知识底座。"准"的根基是一样的，区别在于交互方式和适用场景。

5.1 共同的知识底座

两者都依赖同一套 Data Catalog:

- 同一份 tables_metadata.json 做表定位
- 同一批 tables/*.md 提供业务规则和 AI_HINT
- 同一份 relationships.json 指导跨表关联

分析师的经验只需沉淀一次，两个产品同时受益。

5.2 数据分析 PA：深度分析的系统化流程

PA 是一个流程智能体，由 5 个 Action Agent (AA) 组成，每个 AA 解决一个具体问题：

AA	职责	解决的威胁
AA-01 需求澄清	先探索表结构，再基于事实向用户提问	需求理解偏差
AA-02 SQL 设计	读取 AI_HINT 写 SQL + 独立 sql-checker 验证	业务规则违反
AA-03 数据查询	执行 SQL 并分析结果	分析维度窄
AA-04 报告生成	输出结构化的分析报告	结论表达不清
AA-05 报告更新	增量更新已有报告	重复劳动

PA 有几个独特的设计：

先探索，后澄清。 AA-01 不是直接根据用户描述猜字段，而是先读 Data Catalog 找到匹配表，执行 DESCRIBE TABLE 获取真实表结构，基于事实向用户提问。硬性约束：禁止询问表中不存在的字段，禁止基于假设提问。这防止了一类经典 AI 幻觉——AI 自己"发明"一个字段名来问用户。

sql-checker 独立验证。 AA-02 写完 SQL 后，会交给一个独立的 subagent（拥有 200k 独立 context）做第三方验证：需求和 SQL 逻辑是否一致？字段名是否真实存在？业务规则是否遵守？为什么要独立的 subagent？因为写 SQL 的 AI 会"自我合理化"——它认为自己写对了，验证时也觉得对。独立验证者没有这个包袱。

data_catalog 是硬边界。 分析洞察环节的 AI 有一条铁律：所有补充查询建议，必须基于 data_catalog 中存在的表和字段。不能建议查不存在的数据，只能在已有表的范围内深挖。

适用场景： 复杂分析需求、系统化看板搭建、需要多轮澄清和深度洞察的报告。

5.3 数探：快速对话的即时响应

数探是飞书群里的 AI 员工。@一下，说人话就出结果。

与 PA 相比，数探的特点是：

- **更快：** 对话式交互，一问一答，十几秒出结果
- **更便捷：** 在飞书群里直接用，不用打开新页面
- **带记忆：** 集成了 OpenViking 语义记忆系统，跨会话记住上下文。上次问过的表、用过的口径，这次不用重新说
- **即时可视化：** 查询结果自动生成图表，嵌入飞书卡片

适用场景： 日常查数、快速验证、临时数据需求、"帮我看一眼这个数对不对"。

5.4 怎么选？

需求类型	推荐工具	原因
------	------	----

"eufy 上周美国销售额多少？"	数探	简单直接，秒出结果
"帮我分析各品牌 Q1 转化率趋势并出一份报告"	PA	需要多轮澄清 + 深度分析 + 结构化报告
"这个数据看起来不对，帮我验证一下"	数探	快速验证，对话追问
"搭建一个月度销售看板"	PA	系统化需求，需要完整的分析流程

一个粗略的判断标准：**如果你能一句话说清需求，用数探；如果需要来回讨论才能定义清楚，用 PA。**

六、为什么"准"？——从错误预防的视角

"准"的本质不是做对了什么，而是**系统性地预防了每一类可能的错误。**

6.1 七类典型错误 x 七道防线

错误类型	没有 Data Catalog 时	有 Data Catalog 后
选错表	AI 靠表名猜，30% 概率选错	JSON 索引精准匹配 + exclude_patterns 排除误匹配
字段大小写	brand/country 大小写随机	business_rule_case 强制 brand 小写 country 大写
缺少分区过滤	可能不带日期条件，全表扫描	required_filter 标记日期为必须条件
订单重复计算	用 order_id 去重，跨品牌重复	business_rule_order_dedup 指定三段拼接去重
错误的 JOIN	用 country 关联，丢失跨境订单	relationships.json 标注禁止用 country
业务术语歧义	"会员"可能指付费/普通/全部	ask_user_about_member 触发主动澄清
聚合粒度错误	不清楚表粒度，结果膨胀	granularity 标签明确数据粒度 + 正反示例

6.2 "准"的核心公式

准确率 = 正确的表 x 正确的规则 x 消除歧义的澄清

三者缺一不可。Data Catalog 三者都覆盖了：

- tables_metadata.json 保证找到正确的表
- AI_HINT business_rule 保证遵守正确的规则
- AI_HINT ask_user_about 保证消除关键歧义

七、更深层的思考：一次技术平权

7.1 本质：把分析师的专属能力赋能给所有人

在 AI 介入之前，查数据是数据分析师的专属技能。业务方想看一个数，流程是：

代码块

1 提需求 -> 排队 -> 分析师理解需求 -> 写 SQL -> 跑数 -> 整理结果 -> 交付

一个简单的查询，从提需求到拿到结果，短则半天，长则数天。

现在，业务方在飞书群 @数探 或打开数据分析 PA，用自然语言描述需求，十几秒到几分钟就能获得**和分析师同等质量**的数据服务。

这不是“替代分析师”，而是**释放分析师**。当 80% 的重复取数需求被 AI 承接后，分析师有了时间去做更有价值的事：

- 数据基建：优化表结构、建设指标体系
- 性能提升：查询优化、数据质量监控
- 深度分析：战略洞察、趋势预判、归因分析
- 知识沉淀：维护和完善 Data Catalog 本身

7.2 分析师角色的进化：从“查数工具人”到“知识工程师”

Data Catalog 的建设过程，实际上重新定义了分析师的核心价值。

过去，分析师的价值体现在“会写 SQL”。现在，这个能力已经被 AI 平替了。

分析师**不可被替代的价值是：理解业务规则，并把这种理解编码成 AI 可执行的知识。**

每写一条 AI_HINT，就让所有使用数探和 PA 的人受益。每补一条 ask_user_about，就预防了未来所有用户可能踩的坑。每完善一段 relationships.json，就教会了 AI 一个连表的正确方式。

分析师的最高产出，不再是一份报表，而是一套让 AI 永远不犯同一个错误的规则体系。

7.3 一个值得思考的问题：职能方如何用 AI 服务上下游？

我们做的事情，本质上是**职能方用 AI 服务上下游客户**的一次实践。

不是把 AI 当工具给自己提效（“我用 AI 写 SQL 写得更快了”），而是把自己的专业能力通过 AI 辐射给更多人（“所有人都能像我一样准确地查数据了”）。

这个模式可以复制：

- **财务团队**：把报销规则、预算口径、核算逻辑写成 AI 可读的知识库，让业务方自助查财务数据
- **法务团队**：把合同条款审核规则编码成 AI_HINT，让业务方在提交合同前就能获得初步审核
- **HR 团队**：把薪酬规则、假期政策、入职流程编码成结构化文档，让员工自助查询

核心思路是一样的：把专业知识从人脑搬到文件系统，用 AI 作为放大器，让专业能力突破"一对一服务"的瓶颈。

未来的职能方竞争力 = 专业能力 x AI 放大倍数

谁先把自己的专业知识编码成 AI 可执行的规则，谁就率先完成了这次技术平权。

八、结语

"准"的秘密不在于 AI 有多聪明，而在于我们把"什么是对的"写得有多清楚。

当分析师把自己对字段、表、SQL 的理解，固化成 AI 能懂的语言（Data Catalog + AI_HINT），AI 读取之后，也用用户能懂的语言向用户澄清需求、解释数据——这才是准和体验好的根本原因。

这不只是一个技术项目，而是一次思维方式的转变：

从"我来服务你"到"我教 AI 来服务你"。

分析师的角色没有被削弱，反而被放大了——过去服务 10 个人，现在通过 AI 同时服务整个组织。知识不再锁在某个人的脑子里，而是沉淀在一套可维护、可迭代、可复用的文档体系中。

最好的 AI 应用，不是让 AI 更聪明，而是让人的智慧以 AI 可理解的方式流动起来。

DTC 数据分析团队 / 2026-03